```
class Node:
    def __init__(self, board, parent=None, move=None, depth=0):
        self.board = board
        self.parent = parent
        self.move = move
        self.depth = depth
        self.blank_pos = self.find_blank()

    def find_blank(self):
        for r in range(3):
            for c in range(3):
                if self.board[r][c] == 0:
                    return (r, c)
        return None

    def get_successors(self):
        successors = []
        r, c = self.blank_pos
        deltas = [(-1, 0, "Up"), (1, 0, "Down"), (0, -1, "Left"), (0, 1, "Right")]
        for dr, dc, move_name in deltas:
            nr, nc = r + dr, c + dc
            if 0 <= nr < 3 and 0 <= nc < 3:
                new_board = [row[:] for row in self.board]
                new_board[r][c], new_board[nr][nc] = new_board[nr][nc], new_board[r][c]
                successors.append(Node(new_board, self, move_name, self.depth + 1))
        return successors

    def is_goal(self, goal):
        return self.board == goal

    def get_path(self):
        node, path = self, []
        while node:
            path.append(node.move)
            node = node.parent
        return path[::-1][1:]

def solve_dfs(initial_board, goal_board, max_depth=20):
    start_node = Node(initial_board)
    stack = [start_node]
    visited = set()

    while stack:
        current = stack.pop()
        board_tuple = tuple(tuple(row) for row in current.board)

        if current.is_goal(goal_board):
            path_nodes = []
            node_ptr = current
            while node_ptr:
                path_nodes.append(node_ptr)
                node_ptr = node_ptr.parent
            path_nodes.reverse()

            print("DFS Path (Board States and Moves):")
            for i, node in enumerate(path_nodes):
                print(f"Step {i}: Move: {node.move if node.move else 'Initial State'}")
                for row in node.board:
                    print(row)
                print("-" * 20)
            print(f"Total Path Cost (DFS): {current.depth} moves")
            return current.get_path()

        if board_tuple not in visited:
            visited.add(board_tuple)
            if current.depth < max_depth:
                for succ in reversed(current.get_successors()):
                    succ_tuple = tuple(tuple(row) for row in succ.board)
                    if succ_tuple not in visited:
                        stack.append(succ)
    return None

initial = [[1, 2, 3], [0, 4, 6], [7, 5, 8]]
goal =    [[1, 2, 3], [4, 5, 6], [7, 8, 0]]
solve_dfs(initial, goal)
```

```
--------------------
Step 6: Move: Up
[2, 4, 3]
[1, 5, 0]
[7, 8, 6]
--------------------
Step 7: Move: Left
[2, 4, 3]
[1, 0, 5]
[7, 8, 6]
--------------------
Step 8: Move: Up
[2, 0, 3]
[1, 4, 5]
[7, 8, 6]
--------------------
Step 9: Move: Left
[0, 2, 3]
[1, 4, 5]
[7, 8, 6]
--------------------
Step 10: Move: Down
[1, 2, 3]
[0, 4, 5]
[7, 8, 6]
--------------------
Step 11: Move: Right
[1, 2, 3]
[4, 0, 5]
[7, 8, 6]
--------------------
Step 12: Move: Right
[1, 2, 3]
[4, 5, 0]
[7, 8, 6]
--------------------
Step 13: Move: Down
[1, 2, 3]
[4, 5, 6]
[7, 8, 0]
--------------------
Total Path Cost (DFS): 13 moves
['Up',
 'Right',
 'Down',
 'Down',
 'Right',
 'Up',
 'Left',
 'Up',
 'Left',
 'Down',
 'Right',
 'Right',
 'Down']
```

```
def depth_limited_search(node, goal_board, limit, visited):
    if node.is_goal(goal_board):
        return node
    if node.depth == limit:
        return None

    board_tuple = tuple(tuple(row) for row in node.board)

    visited.add(board_tuple)
    for succ in node.get_successors():
        succ_tuple = tuple(tuple(row) for row in succ.board)
        if succ_tuple not in visited:
            result = depth_limited_search(succ, goal_board, limit, visited)
            if result is not None:
                return result
    visited.remove(board_tuple)
    return None

def solve_ids(initial_board, goal_board, max_limit=30):
    for limit in range(max_limit + 1):
        visited = set()
        start_node = Node(initial_board)
        result = depth_limited_search(start_node, goal_board, limit, visited)
        if result is not None:

            path_nodes = []
            node_ptr = result
            while node_ptr:
                path_nodes.append(node_ptr)
                node_ptr = node_ptr.parent
            path_nodes.reverse()
```

```
            print(f"\nIDS Path (Board States and Moves) at limit {limit}:")
            for i, node in enumerate(path_nodes):
                print(f"Step {i}: Move: {node.move if node.move else 'Initial State'}")
                for row in node.board:
                    print(row)
                print("-" * 20)
            print(f"Total Path Cost (IDS): {result.depth} moves")
            return result.get_path()
    return None


solve_ids(initial, goal)
```

```
IDS Path (Board States and Moves) at limit 3:
Step 0: Move: Initial State
[1, 2, 3]
[0, 4, 6]
[7, 5, 8]
--------------------
Step 1: Move: Right
[1, 2, 3]
[4, 0, 6]
[7, 5, 8]
--------------------
Step 2: Move: Down
[1, 2, 3]
[4, 5, 6]
[7, 0, 8]
--------------------
Step 3: Move: Right
[1, 2, 3]
[4, 5, 6]
[7, 8, 0]
--------------------
Total Path Cost (IDS): 3 moves
['Right', 'Down', 'Right']
```